

Final Docker & Integration Plan

1. Docker Integration Goal

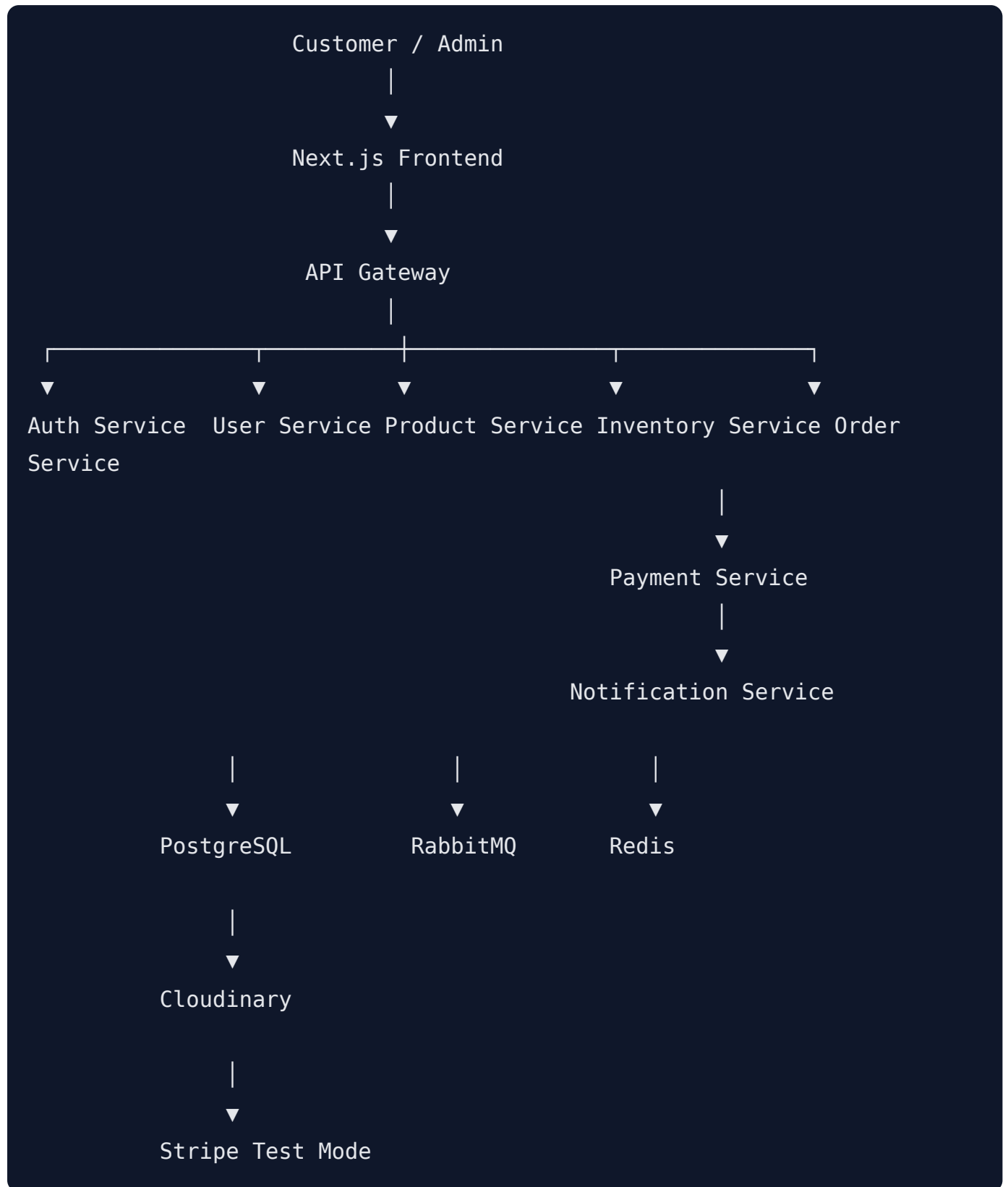
Containerize the complete Clothing E-Commerce Platform so that the frontend, backend microservices, database, and infrastructure services run together in an isolated environment using Docker Compose.

Each component will run independently while communicating over Docker's internal network. The application should be started with a single command and be easy for every team member to run locally.

2. Docker Technology Stack

Purpose	Technology
Containerization	Docker
Multi-Container Management	Docker Compose
Frontend	Next.js
Backend	Node.js + Express.js
API Gateway	Express Gateway
Database	PostgreSQL
ORM	Prisma ORM
Authentication	JWT
Message Broker	RabbitMQ
Cache	Redis
Image Storage	Cloudinary
Payment	Stripe Test Mode
API Documentation	Swagger / OpenAPI

3. Complete Integration Architecture



4. Services to Containerize

The application consists of the following Docker containers:

Container	Purpose
frontend	Next.js Customer + Admin
api-gateway	Route requests to microservices
auth-service	Authentication & JWT
user-service	User Profile, Address, Wishlist
product-service	Products, Categories, Cloudinary
inventory-service	Stock & Inventory
order-service	Cart & Orders
payment-service	Stripe Payment
notification-service	Email & Notifications
postgres	Database
rabbitmq	Event Communication
redis	Cache

Total Containers: 12

5. Docker Networking

Docker Compose creates a shared internal network.

All services communicate using service names.

Example:

```
frontend
↓
api-gateway
↓
product-service
↓
postgres
```

No service communicates using localhost.

6. Frontend Integration

Frontend communicates only with the API Gateway.

Flow

```
Customer/Admin
↓
Next.js
↓
API Gateway
↓
Backend Services
```

The frontend never connects directly to:

- PostgreSQL

- Prisma
 - RabbitMQ
 - Redis
-

7. API Gateway Integration

Responsibilities

- Receive frontend requests
- Verify JWT
- Forward requests
- Route APIs
- Error handling
- Logging

Routes

```
/api/auth  
  
/api/users  
  
/api/products  
  
/api/inventory  
  
/api/orders  
  
/api/payments  
  
/api/notifications
```

8. Auth Service Integration

Responsibilities

- Register
- Login
- JWT
- Refresh Token
- Email Verification
- Forgot Password
- Reset Password

Database Tables

- users
- refresh_tokens
- email_verification_tokens
- password_reset_tokens

Publishes Event

UserRegistered

9. User Service Integration

Responsibilities

- Profile
- Address
- Wishlist

Database Tables

- users
- addresses
- wishlist_items

Consumes

UserRegistered

10. Product Service Integration

Responsibilities

- Categories
- Products
- Images

Database Tables

- categories
- products
- product_images

Cloudinary Flow

Admin

↓

Upload Image

↓

Cloudinary

↓

Image URL

↓

Database

11. Inventory Service Integration

Responsibilities

- Product Stock
- Size
- Color
- SKU
- Stock Updates

Database Table

product_variants

Publishes

InventoryReserved

InventoryReleased

Consumes

OrderPlaced

OrderCancelled

12. Order Service Integration

Responsibilities

- Cart
- Checkout
- Orders
- Tracking

Database Tables

- carts

- cart_items
- orders
- order_items

Publishes

OrderPlaced

OrderCancelled

Consumes

InventoryReserved

PaymentConfirmed

PaymentFailed

13. Payment Service Integration

Responsibilities

- Stripe Payment
- Refund
- Webhook

Database Tables

- payments
- refunds

Publishes

PaymentConfirmed

PaymentFailed

RefundCompleted

14. Notification Service Integration

Responsibilities

- Email Notifications
- Notification History

Database Table

notifications

Consumes

UserRegistered

OrderPlaced

PaymentConfirmed

PaymentFailed

ShipmentUpdated

RefundCompleted

15. PostgreSQL Integration

PostgreSQL stores all persistent business data.

Connected Services

- Auth
- User
- Product
- Inventory
- Order
- Payment
- Notification

Prisma handles

- Models
- Relations
- Migrations
- CRUD Operations

16. RabbitMQ Integration

RabbitMQ handles asynchronous communication.

Core Events

UserRegistered

OrderPlaced

OrderCancelled

InventoryReserved

InventoryReleased

PaymentConfirmed

PaymentFailed

ShipmentUpdated

RefundCompleted

RabbitMQ will contain:

- Exchanges
- Queues
- Routing Keys
- Dead Letter Queue (DLQ)

17. Redis Integration

Redis caches frequently accessed data.

Used For

- Product Listing
- Categories
- Popular Products

Flow

Frontend

↓

Product API

↓

Redis

↓

MISS

↓

PostgreSQL

↓

Cache

↓

Return

18. Cloudinary Integration

Flow

Admin



Frontend



Product Service



Cloudinary



Image URL



Database

Only image URLs and Public IDs are stored in PostgreSQL.

19. Stripe Integration

Flow

Checkout

↓

Payment Service

↓

Stripe

↓

Webhook

↓

Payment Confirmed

↓

RabbitMQ

↓

Notification Service

20. Environment Variables

Each service contains its own `.env`.

Examples

DATABASE_URL

JWT_SECRET

JWT_REFRESH_SECRET

REDIS_URL

RABBITMQ_URL

CLOUDINARY_CLOUD_NAME

CLOUDINARY_API_KEY

CLOUDINARY_API_SECRET

STRIPE_SECRET_KEY

STRIPE_WEBHOOK_SECRET

EMAIL_USER

EMAIL_PASSWORD

21. Docker Folder Structure


```
clothing-ecommerce/
|
├─ frontend/
|   └─ Dockerfile
|
├─ backend/
|   ├─ api-gateway/
|   |   └─ Dockerfile
|   ├─ auth-service/
|   |   └─ Dockerfile
|   ├─ user-service/
|   |   └─ Dockerfile
|   ├─ product-service/
|   |   └─ Dockerfile
|   ├─ inventory-service/
|   |   └─ Dockerfile
|   ├─ order-service/
|   |   └─ Dockerfile
|   ├─ payment-service/
|   |   └─ Dockerfile
|   ├─ notification-service/
|   |   └─ Dockerfile
|   └─ shared/
|
├─ database/
|   ├─ prisma/
|   ├─ sql/
|   └─ README.md
|
├─ docker/
|   ├─ postgres/
|   |   └─ init.sql
|   ├─ rabbitmq/
|   |   ├─ rabbitmq.conf
|   |   ├─ definitions.json
|   |   └─ README.md
|   └─ redis/
```

```
| | └─ redis.conf
| | └─ README.md
| └─ scripts/
| | └─ start.sh
| | └─ wait-for-db.sh
| | └─ healthcheck.sh
| └─ README.md
|
└─ docs/
|
└─ docker-compose.yml
└─ .dockerignore
└─ .env.example
└─ README.md
```

22. Docker Compose Responsibilities

Docker Compose will:

- Create Docker network
- Start PostgreSQL
- Start Redis
- Start RabbitMQ
- Start Backend Services
- Start API Gateway
- Start Frontend
- Mount volumes
- Pass environment variables
- Restart failed services
- Run health checks

23. Startup Order

```
PostgreSQL
  ↓
Redis
  ↓
RabbitMQ
  ↓
Auth Service
  ↓
User Service
  ↓
Product Service
  ↓
Inventory Service
  ↓
Order Service
  ↓
Payment Service
  ↓
Notification Service
  ↓
API Gateway
  ↓
Frontend
```

24. Integration Testing Order

```
graph TD; 1[1. PostgreSQL Connection] --> 2[2. Prisma Migration]; 2 --> 3[3. Redis Connection]; 3 --> 4[4. RabbitMQ Connection]; 4 --> 5[5. Auth Service]; 5 --> 6[6. User Service]; 6 --> 7[7. Product Service]; 7 --> 8[8. Inventory Service]; 8 --> 9[9. Order Service]; 9 --> 10[10. Payment Service]; 10 --> 11[11. Notification Service]; 11 --> 12[12. API Gateway]; 12 --> 13[13. Frontend]; 13 --> 14[14. Cloudinary Upload]; 14 --> 15[15. Stripe Payment]; 15 --> 16[16. RabbitMQ Events]; 16 --> 17[17. Redis Cache]; 17 --> 18[18. Complete Checkout Flow];
```

1. PostgreSQL Connection
- ↓
2. Prisma Migration
- ↓
3. Redis Connection
- ↓
4. RabbitMQ Connection
- ↓
5. Auth Service
- ↓
6. User Service
- ↓
7. Product Service
- ↓
8. Inventory Service
- ↓
9. Order Service
- ↓
10. Payment Service
- ↓
11. Notification Service
- ↓
12. API Gateway
- ↓
13. Frontend
- ↓
14. Cloudinary Upload
- ↓
15. Stripe Payment
- ↓
16. RabbitMQ Events
- ↓
17. Redis Cache
- ↓
18. Complete Checkout Flow

25. Complete Application Flow

```
Customer/Admin
  ↓
Next.js Frontend
  ↓
API Gateway
  ↓
Backend Services
  ↓
PostgreSQL
  ↓
Stripe Payment
  ↓
RabbitMQ
  ↓
Notification Service
  ↓
Email / In-App Notification
```

26. Final Deliverables

The Docker & Integration module will deliver:

- Dockerfile for Frontend
- Dockerfile for each Backend Service
- Docker Compose configuration
- PostgreSQL container
- RabbitMQ container
- Redis container
- Shared Docker network
- Environment variable configuration
- Service health checks
- Frontend–Backend integration

- Database integration
 - Cloudinary integration
 - Stripe integration
 - RabbitMQ event communication
 - Redis caching
 - Complete application startup using Docker Compose
-

Final Review

This Docker & Integration Plan is fully aligned with:

- **Frontend Plan** (Customer + Admin pages, JWT authentication, API Gateway, Cloudinary, Stripe)
- **Backend Plan** (API Gateway, Auth, User, Product, Inventory, Order, Payment, Notification microservices)
- **Database Plan** (17 core tables, PostgreSQL + Prisma, RabbitMQ, Redis, Cloudinary, Stripe)

It is suitable as the final implementation plan for your assignment without introducing unnecessary enterprise-level complexity.